Smaller Universal Reversible Turing Machines and the Nonexistence of Two-State Ones

Tetsuo Yokoyama
Nanzan University

October 2026
Extended version

Abstract

A reversible Turing machine (RTM) is a Turing machine in which every instantaneous description has at most one predecessor. Morita's book (2017) lists nine universal RTMs (URTMs); we write URTM(m, n) for one with m states and n symbols. We construct seven URTMs that are not dominated by any of these nine: URTM$(3, 600)$, URTM$(4, 42)$, URTM$(5, 39)$, URTM$(6, 30)$, URTM$(7, 28)$, URTM$(8, 21)$, and URTM$(85, 2)$. Together with six of Morita's machines, they give 13 non-dominated points. The new machines come from improved versions of Morita's conversions to three states, to four states, and to two symbols. We use a numbering of states by the direction of entry, counting in the reverse direction, a block decomposition of state numbers that trades states for symbols, and the merging of symbols that are never read or written in the same context. For example, the improved conversion to three states uses $O(mn^2)$ symbols instead of $O(m^2n^3)$, and it turns URTM$(32, 3)$ into URTM$(3, 600)$ instead of URTM$(3, 36654)$. On the negative side, we show that the halting problem of RTMs with at most two states is decidable for finite instantaneous descriptions, by a single procedure that also takes the machine as input. Consequently, there is no URTM with at most two states, which answers a question left open by Morita. We also show that no three-state, two-symbol RTM is universal. All these results are formalized in Lean 4 with Mathlib. Universality is formalized after Morita's definition, with computable encoding and decoding functions, which we give explicitly for all 13 points.

1 Introduction

Reversible computing concerns computations in which each step uniquely determines its predecessor. Bennett showed that irreversible computations can be simulated reversibly without loss of computational power [2]. A reversible Turing machine (RTM) is one of the basic models of reversible computing, and a universal RTM is called a URTM. As for ordinary universal Turing machines (UTMs) [15, 7, 13, 16], a natural question is how small a URTM can be. We measure the size by the numbers of states and symbols, and write URTM(m, n) for a URTM with m states and n symbols.

Morita's book [10] lists nine URTMs. Six of them, URTM$(10, 8)$, $(13, 7)$, $(15, 6)$, $(17, 5)$, $(24, 4)$, and $(32, 3)$, simulate cyclic tag systems with halting condition directly; URTM$(17, 5)$ is due to Morita and Yamaguchi. The other three, URTM$(4, 168)$, $(138, 2)$, and $(3, 36654)$, are obtained from them by general conversions that reduce the number of states or of symbols. The book remarks that these three leave room for reduction, and it states that the existence of a two-state URTM is open. In a later survey, after recalling that two-state TMs are universal in the irreversible case, Morita writes that "it is unknown whether the class of two-state RTMs is universal" [11].

Contributions.

- *New upper bounds* (Section 3). We give seven URTMs, $(3, 600)$, $(4, 42)$, $(5, 39)$, $(6, 30)$, $(7, 28)$, $(8, 21)$, and $(85, 2)$, none of which is dominated by Morita's nine points (Figure 4). The three-state machine improves the conversion of Morita's Theorem 5.6, the machines with four to eight states come from a block decomposition of the conversion of Theorem 5.5 followed by symbol merging, and the two-symbol machine optimizes the conversion of Theorem 5.4. No conversion in the book yields five to nine states; the block decomposition gives a whole family of state-symbol trade-offs.
- *Universality in the sense of Morita's definition* (Section 4). Morita's Definition 7.1 asks for total recursive encoding and decoding functions, but the book does not give them for its URTMs. We give computable encodings and decodings for all 13 non-dominated points and prove universality in this sense.
- *Nonexistence for two states* (Section 5). The halting problem of RTMs with at most two states is decidable for finite instantaneous descriptions, uniformly in the machine. Hence no URTM with at most two states exists, for any number of symbols. The key fact is that, in a two-state RTM, the state equals the direction of the last move, so a half-line of the tape can tell the rest of the tape only that the head has come back.
- *Nonexistence for three states and two symbols* (Section 6). No three-state, two-symbol RTM is universal.
- *Formalization* (Section 7). All theorems above are formalized and proved in Lean 4 [8] with Mathlib [9], using only the standard axioms. The formalized machine model reproduces the halting step counts of the published test runs of all nine of Morita's machines.

For each number of states, the constructions give an upper bound on the minimum number of symbols of a URTM. We do not claim that any of the new points is minimal.

Related work. Small UTMs have been studied for a long time; Woods and Neary survey the results [16]. The non-dominated small UTMs listed in Morita's book include UTM$(2, 18)$ and UTM$(4, 6)$ of Rogozhin [15], UTM$(3, 9)$ of Kudlek and Rogozhin [7], and UTM$(5, 5)$, $(6, 4)$, $(9, 3)$, and $(15, 2)$ of Neary and Woods [13]. These machines need not be reversible, and their conventions on halting and on stationary moves differ from those used here. The URTMs of Morita and his coauthors are collected in [10, Ch. 7], together with the conversions that we improve. Turing machines and universal Turing machines have been formalized in Isabelle/HOL [17], Matita [1], and Coq [6], and Mathlib contains a universal partial recursive function [3]. These formalizations treat general constructions rather than small machines, and none of them is about reversible computation. Closest to our two-state result is Dudenhefner's decision procedure, certified in Coq, for the halting problem of reversible two-counter machines with a particular instruction set; it contrasts with Morita's result that reversible two-counter machines with another instruction set are universal [5]. To our knowledge, small URTMs and their universality in the sense of Definition 7.1 have not been formalized before.

2 Preliminaries

We follow Morita's book [10] and state the definitions in the form used in the formalization.

Definition 1 (Machines and runs)**.** A *quintuple* $r = [p, s, t, D, q]$ consists of a source state p, a read symbol s, a written symbol t, a direction $D \in \{\mathrm{L}, \mathrm{Z}, \mathrm{R}\}$, and a target state q, where $p, s, t, q \in \mathbb{N}$. The directions L, Z, and R move the head by -1, 0, and $+1$; a quintuple with direction Z is a *zero-shift*. A Turing machine (TM) is a finite list T of quintuples; only membership in T matters. An *instantaneous description* (ID) is a triple $(st, pos, tape)$ with $st \in \mathbb{N}$, $pos \in \mathbb{Z}$,

and $\mathit{tape} : \mathbb{Z} \to \mathbb{N}$; the symbol 0 is the blank. A quintuple r applies to an ID α if $r.p = \alpha.st$ and $r.s = \alpha.\mathit{tape}(\alpha.\mathit{pos})$; the result writes $r.t$ at $\alpha.\mathit{pos}$, moves the head by the displacement of $r.D$, and enters $r.q$. An ID is *halting* for T if no quintuple of T applies to it. We write $\mathrm{Halts}_T(\alpha)$ if T reaches a halting ID from α in finitely many (possibly zero) steps.

Definition 2 (Determinism and reversibility)**.** T is *deterministic* if any two quintuples of T with the same source state and read symbol are equal. T is *reversible* if any two distinct quintuples of T with the same target state have the same direction and different written symbols.

This is Morita's Definition 5.4, specialized to the quintuple form. By reversibility, all quintuples entering a state move in the same direction (the unique shift-direction property, [10, Lemma 5.5]). For an RTM T we write Q_+ and Q_- for the sets of states entered by right moves and by left moves, and $m_+ = |Q_+|$, $m_- = |Q_-|$.

Definition 3 (Size)**.** $\mathrm{NStates}(T)$ is the number of distinct source states of T, and $\mathrm{NSyms}(T)$ is the number of elements of $\{0\} \cup \{r.s, r.t \mid r \in T\}$. A point (m, n) *dominates* (m', n') if $m \le m'$, $n \le n'$, and $(m, n) \neq (m', n')$.

Halting states, which have no outgoing quintuples, are not counted, as in Morita's book (Section 4.2).

Codes of finite IDs. An ID is *finite* if it has finitely many non-blank cells. Let $\mathit{Code} = \mathbb{N} \times \mathrm{List}\,\mathbb{N} \times \mathrm{List}\,\mathbb{N}$. For $p \in \mathbb{Z}$ and $(q, l, r) \in \mathit{Code}$, the ID $\mathrm{toID}(p, (q, l, r))$ has state q and head position p, carries r on the cells $p, p+1, \ldots$ and l on the cells $p-1, p-2, \ldots$, and is blank elsewhere. For an ID α, $\alpha.\mathit{pos}$ is its head position. $\mathrm{IsIDOf}(T, \alpha)$ holds if α is finite, its state occurs in T, and every cell holds the blank or a symbol occurring in T. The enumeration $\mathrm{tm}(i)$, $i \in \mathbb{N}$, decodes i into a list of quintuples by the standard bijection between $\mathbb{N}$ and lists of tuples of natural numbers in Mathlib, and replaces a non-deterministic result by the empty list.

Definition 4 (Universality, after [10, Def. 7.1])**.** A table U satisfies $\mathrm{IsUTMStrict}(U)$ if there are an encoding $c : \mathbb{N} \times \mathit{Code} \to \mathit{Code}$ and a decoding $d : \mathit{Code} \to \mathrm{Option}\,\mathit{Code}$, both computable, such that for all $i \in \mathbb{N}$ and all $a \in \mathit{Code}$ with $\mathrm{IsIDOf}(\mathrm{tm}(i), \mathrm{toID}(0, a))$:

1. $\mathrm{IsIDOf}(U, \mathrm{toID}(0, c(i, a)))$;
2. if $\mathrm{tm}(i)$ reaches a halting ID β from $\mathrm{toID}(0, a)$, then U reaches a halting ID γ from $\mathrm{toID}(0, c(i, a))$, γ has a code (some g with $\mathrm{toID}(\gamma.\mathit{pos}, g) = \gamma$), and every such code g satisfies $d(g) = \mathrm{some}(b)$ for some b with $\mathrm{toID}(\beta.\mathit{pos}, b) = \beta$;
3. if $\mathrm{tm}(i)$ does not halt from $\mathrm{toID}(0, a)$, then U does not halt from $\mathrm{toID}(0, c(i, a))$.

The weaker predicate $\mathrm{IsUTM}(U)$ drops condition 1 and requires the decoding in condition 2 only for some code of γ.

IsUTMStrict implies IsUTM. We use the strong form for existence and the weak form for nonexistence whenever possible, so both kinds of results hold under either form. The requirement in condition 2 that *every* code of γ decodes correctly matters: a code can be padded with trailing blanks without changing the ID it denotes, so a condition on *some* code alone is weak. Morita's IDs are strings $\alpha q \beta$ without a head position, whereas our IDs are two-way infinite tapes; we prove in Lean that, for deterministic tables, IsUTMStrict implies the formulation on string IDs.

Definition 5 (Cyclic tag system with halting condition [10, Def. 7.5])**.** A CTSH is $\hat{C} = (k, (\mathrm{halt}, w_1, \ldots, w_{k-1}))$ with $k \ge 1$ and $w_j \in \{Y, N\}^*$. An ID is a pair (v, j) of a string $v \in \{Y, N\}^*$ and a phase $j \in \{0, \ldots, k-1\}$. The transitions are $(Yv, j) \Rightarrow (vw_j, j+1 \bmod k)$ for $j \neq 0$ and $(Nv, j) \Rightarrow (v, j+1 \bmod k)$. The IDs $(Yv, 0)$ and (λ, j) are halting.

Morita's six original machines simulate an arbitrary CTSH started from $(v, 0)$, and CTSHs can simulate arbitrary TMs through 2-tag systems [4], [10, Sect. 7.2].

Table 1: Conversions and their effect on the numbers of states and symbols

Conversion	States	Symbols	Example	Source
Reducing states (symbols increase)				
To four states	4	$(2m+1)n$	$(10,8) \to (4,168)$	[10, Thm. 5.5]
Disjoint ranges by direction	4	$(m+1)n$	$(10,8) \to (4,88)$	Sect. 3.1
Reverse counting	4	$\le (\max(m_+,m_-)+1)n$	$(10,8) \to (4,48)$	Sect. 3.1
Blocks and merging of roles	4–9	$n+\sum_{i\in V}\lvert X_i\rvert$	$(15,6) \to (7,32)$	Sect. 3.2
To three states	3	$N(2M-1)$	$(32,3) \to (3,36654)$	[10, Thm. 5.6]
One symbol per cell, shared counters	3	$O(mn^2)$	$(32,3) \to (3,600)$	Sect. 3.4
Reducing symbols (states increase)				
To two symbols	$\le mk+P_r+P_w$	2	$(24,4) \to (138,2)$	[10, Thm. 5.4]
Code optimization, state merging	per table	2	$(13,7) \to (85,2)$	Sect. 3.5
Same number of states				
Symbol merging	unchanged	number of colors	$(4,50) \to (4,42)$	Sect. 3.3

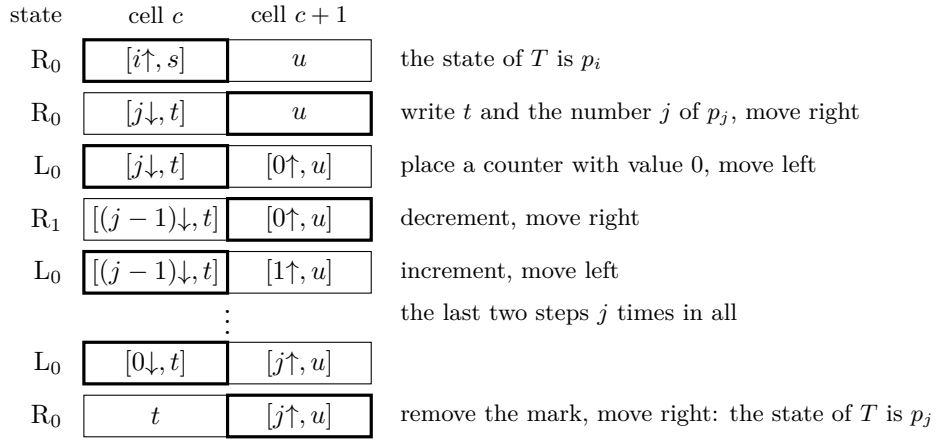


Figure 1: Morita's four-state conversion executing a right-moving quintuple $[p_i, s, t, \mathrm{R}, p_j]$

3 Constructions

All our machines are obtained from Morita's six original machines by conversions. Table 1 summarizes how each conversion changes the numbers of states and symbols. Throughout, m and n are the numbers of states and symbols of the original RTM T.

3.1 Four states: numbering by direction and reverse counting

Morita's conversion to four states [10, Sect. 5.3.4] keeps the number of the current state of T on the tape, as a counter attached to the symbol under the head. To execute a quintuple $[p, s, t, D, q]$, the converted machine writes the counter of q into the current cell, places a counter with the initial value in the target cell, and then moves back and forth between the two cells, decrementing the counter of the source cell and incrementing that of the target cell, until the whole number has been carried over. To distinguish the source cell from the target cell, each counter carries an up mark or a down mark. Hence the converted machine has $(2m+1)n$ symbols: the n plain symbols and $2mn$ marked counter symbols. From URTM$(10,8)$ it gives URTM$(4,168)$. The converted machine has the four states $\mathrm{L}_0, \mathrm{L}_1, \mathrm{R}_0, \mathrm{R}_1$. Figure 1 shows how it executes a right-moving quintuple; each row gives the state and the two cells, and the thick frame marks the head. The execution takes $2j+3$ steps, and a left-moving quintuple is executed in the same way with left and right exchanged.

Disjoint ranges by direction. Number the states of Q_+ by $1, \ldots, m_+$ and those of Q_- by $m_+ + 1, \ldots, m$. The converted machine has the four states H_+, H_-, A_+, A_-, where the index is the direction by which the state is entered, and its symbols are the plain symbols s and the

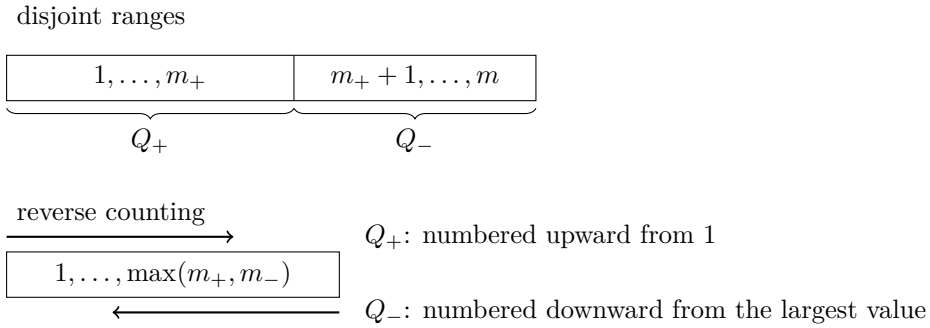


Figure 2: Counter values of the states of Q_+ and Q_- in the four-state conversions

counter symbols $[i, s]$. A *stable* ID has exactly one counter symbol, under the head, and the machine in an H-state; it represents the ID of T obtained by replacing $[i, s]$ by s and taking the state numbered i. For a quintuple $[p, s, t, D, q]$ of T with $p \in Q_e$, the converted machine uses the rules

	state	read	write	move	next
R1	H_e	$[\nu(p), s]$	$[\nu(q), t]$	D	H_D
R2	H_D	s'	$[\ell_D, s']$	$\bar{D}$	$A_{\bar{D}}$
R3	$A_{\bar{D}}$	$[i, x]$, $\ell_D < i \le h_D$	$[i-1, x]$	D	A_D
R4	$A_{\bar{D}}$	$[\ell_D, x]$	x	D	H_D
R5	A_D	$[j, s']$, $\ell_D \le j < h_D$	$[j+1, s']$	$\bar{D}$	$A_{\bar{D}}$

where ν is the numbering, $[\ell_D, h_D]$ is the range of numbers of Q_D, and $\bar{D}$ is the direction opposite to D. The sum of the two counters is invariant during the back-and-forth phase, and R4 fires exactly when the whole number has been carried, so a stable ID is restored after $3 + 2(\nu(q) - \ell_D)$ steps. The state A_+ acts as the target side of a right move and as the source side of a left move, but the counter values it reads in the two roles lie in disjoint ranges, so the table is deterministic; reversibility follows by the same separation together with the reversibility of T. The marks are no longer needed, and the number of symbols becomes $(m + 1)n$. From URTM$(10, 8)$ this gives four states and 88 symbols.

Reverse counting. If the counters are counted in opposite directions for the two directions of motion (the sentinel of Q_+ at the smallest value and that of Q_- at the largest), then the decrement rule R3 of one direction and the increment rule R5 of the other become literally the same quintuple. The two directions can then share one range of values, and the number of symbols is at most $(\max(m_+, m_-) + 1)n$. Figure 2 compares the two numberings. From URTM$(10, 8)$ this gives a reversible machine with four states and 48 symbols. We prove that it simulates an arbitrary CTSH, but not that it is universal in the sense of Definition 4; it is dominated by URTM$(4, 42)$ in any case.

3.2 Block decomposition and merging of roles

Split the number of a state into a pair (block, value within the block). The finite control holds the block and the tape holds only the value, so a counter symbol $[i, s]$ does not record the block, and different blocks reuse the same values. With b_+ blocks for Q_+ and b_- blocks for Q_-, each block β has three roles: H_β (stable), O_β (the source side of a carry), and D_β (the target side of a carry). The rules are those of Section 3.1 with block indices; R1 goes from $H_{\mathrm{blk}(p)}$ to $H_{\mathrm{blk}(q)}$. The roles still have to know their block in the back-and-forth phase, because R4 must enter the correct H-state; this is what costs states. Figure 3 shows the splitting for seven states in two blocks.

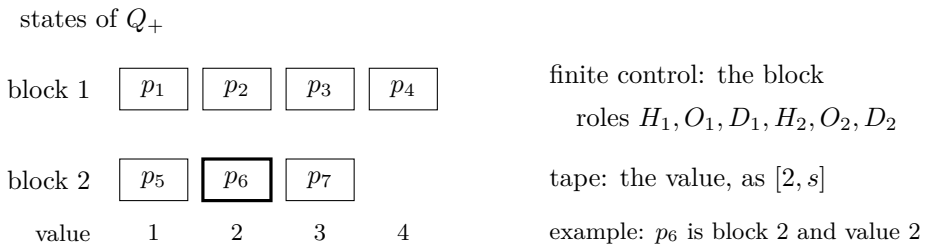


Figure 3: Splitting the numbers of the states of Q_+ into blocks and values (an example with seven states)

Table 2: Machines obtained by block decomposition and merging of roles

Point	Original	V	$\lvert X_i\rvert$ for $i \in V$	$n + \sum \lvert X_i\rvert$	$(\lvert V\rvert + 1)n$
$(4, 46)$	$(10, 8)$	$\{1, 2, 3, 4, 6\}$	8, 8, 8, 8, 6	46	48
$(5, 45)$	$(15, 6)$	$\{1, \ldots, 8\}$	3, 5, 5, 5, 5, 5, 5, 6	45	54
$(6, 37)$	$(10, 8)$	$\{1, 2, 5, 6\}$	6, 7, 8, 8	37	40
$(7, 32)$	$(15, 6)$	$\{1, \ldots, 5\}$	6, 5, 5, 5, 5	32	36
$(8, 28)$	$(17, 5)$	$\{2, 4, 5, 6, 7\}$	3, 5, 5, 5, 5	28	30
$(9, 27)$	$(15, 6)$	$\{1, 2, 3, 4\}$	6, 5, 5, 5	27	30

A *merging of roles* is a partition of the roles into classes; each class becomes one state. Roles in the same class must be entered from the same direction, and the merged table must remain deterministic and reversible. The reverse-counting trick of Section 3.1 is the special case where the source role of one direction and the target role of the other are merged in a single block. A further merging that turned out to be useful puts a single state into a block of its own, gives it a counter value outside the range of the main blocks, and merges its H-role into a back-and-forth state of a main block; then a counter value outside the main range alone signals that the machine is in that state, and no carry is needed.

Only the counter symbols that can actually occur are kept. If V is the set of counter values used and X_i is the set of original symbols that can carry the value i, the number of symbols is

$$n + \sum_{i \in V} |X_i| \;\le\; (|V| + 1)n.$$

The choice of the blocks, of the values, and of the merging is a combinatorial optimization problem. We solved it with a constraint programming model (one Boolean variable for each state, block, and value, with determinism and reversibility as constraints on the generated rules) and with simulated annealing over the partitions. This gives URTM$(4, 46)$ from $(10, 8)$, for example, with $V = \{1, 2, 3, 4, 6\}$: the main blocks use the values 1 to 4, and two states placed in blocks of their own share the value 6. Table 2 lists the six machines with four to nine states obtained in this way. They are all dominated by the machines of Section 3.3, but they are universal in their own right, and they are the inputs to symbol merging. The book contains no conversion that produces five to nine states.

Remark 6. Three states cannot be reached in this framework, which merges neither two H-roles nor the H- and D-roles of the same block. If Q_- has at least two states, then either some block of Q_- has two or more states, and its H- and D-roles fall into different classes, or there are two single-state blocks with two H-roles. In either case the roles entered from the left need at least two classes. The same holds for Q_+, so at least four states are needed for all six of Morita's machines.

Table 3: Machines obtained by symbol merging

Point	Original	Blocks (b_+, b_-)	Symbols before merging	Symbols after merging
$(4, 42)$	$(13, 7)$	$(2, 2)$	50	42
$(5, 39)$	$(13, 7)$	$(3, 2)$	49	39
$(6, 30)$	$(13, 7)$	$(3, 3)$	54	30
$(7, 28)$	$(13, 7)$	$(4, 3)$	50	28
$(8, 21)$	$(13, 7)$	$(4, 4)$	40	21

3.3 Symbol merging

In a converted table, each counter symbol is read by few states and written into few states. Kudlek and Rogozhin reduced a standard UTM by identifying the blank with another symbol [7]. We apply the same idea systematically. Two symbols x and y *conflict* if some state reads both, some state is entered by quintuples writing x and y, or both can occur on the tape of a halting ID reached from an encoded input. Merging pairwise non-conflicting symbols is a coloring of the conflict graph: symbols of the same color become one symbol. The merged table is deterministic and reversible if the original is, and it follows the run of the original step by step.

The general statement that we prove is the following.

Theorem 7 (Symbol merging)**.** *Let U satisfy* IsUTMStrict(U)*, and assume that every halting ID reached from an encoded input is in a state that is not a source state and has all its tape symbols in a fixed finite set F. Let $h, \rho : \mathbb{N} \to \mathbb{N}$ be primitive recursive with $h(0) = \rho(0) = 0$ and $\rho(h(x)) = x$ for every $x \in F$. If the table $h(U)$, obtained by replacing the read and written symbols of every quintuple by their images under h, is deterministic, then* IsUTMStrict$(h(U))$*.*

Here h is the coloring and ρ recovers the original symbols on halting tapes; the hypothesis on the halting states holds because, in runs from encoded inputs, our block tables halt only by entering a halting state. Applied to the fixed tables of Table 2, merging removes one to three symbols. Much more is gained by choosing the blocks and values again, by simulated annealing, with the number of colors after merging as the objective. All five best results come from URTM$(13, 7)$ (Table 3). We do not claim minimality, either of the coloring or over the choices of blocks.

3.4 Three states

Morita's conversion to three states [10, Sect. 5.3.5] has two stages. Lemma 5.9 turns T into a *one-way-updating* RTM $\hat{T}$, which rewrites the tape only while sweeping to the right and returns to the left end in a single state without rewriting. Its tape packs two cells of T into one cell, with $N = mn^2 + n^2 + 1$ symbols (the blank, the pairs, and the pairs marked with a state), and it has $M = m_+n + 2n + 2$ states. Lemma 5.10 then represents the state q_i of $\hat{T}$ by a counter $[x, i]$ with an up or a down mark on the symbol under the head and uses three states L, R_0, R_1, which gives $N(2M - 1)$ symbols. From URTM$(32, 3)$ this is URTM$(3, 36654)$. Of these symbols, 85% are counters that combine a marked pair, one of the m_+n states of $\hat{T}$ entered after a right move, and an up or down mark. We reduce the count in four steps.

Shared counters. In a one-way-updating machine all carries go to the right, so each of the three states always reads counters on a fixed side: L only on the source side, R_0 and R_1 only on the target side. Hence the up and down marks can be identified without losing determinism, and reversibility is preserved, since the written symbols of the rules entering each state remain distinct. The count drops from $N(2M - 1)$ to NM, that is, from 36,654 to 18,476 for $(32, 3)$.

Pruned counters. A counter $[x, k]$ is needed only if some state with number at least k can read x or is entered by writing x. Keeping only these counters gives $N + \sum_x \mathrm{need}(x)$ symbols, where $\mathrm{need}(x)$ is the largest such number. The sum depends on the numbering of the states of $\hat{T}$; states with the same set of relevant symbols can be kept adjacent, and the best order of these groups gives 1,820 symbols for $(32, 3)$. The machines with 18,476 and 1,820 symbols only measure the effect of these two steps: we count their symbols but do not prove that they simulate T, and for the second one we do not show that the kept counters suffice along every run. Universality is proved for the machines of the next two steps.

One symbol per cell. The remaining cost lies in the n^2 pairs and the mn^2 marked pairs. Morita packs two cells to distinguish the blank of T from the blank of $\hat{T}$ and to extend the used region one cell at a time. We instead keep one symbol per cell and represent the blank of T by a separate symbol B inside the used region. The sweep shifts the whole content of T one cell to the right, as in Morita's construction, so a left move of T does not overshoot its target. The current state p of T is recorded as a mark on one cell: on the head cell if $p \in Q_-$, and on the cell to the left of the head if $p \in Q_+$. The one-way-updating machine has $n + 3 + |R| + |G|$ states, where R is the set of pairs (q, t) such that T has a right-moving quintuple into q writing t, and $G = R \cup (\{p_0\} \times S)$ if the initial state p_0 is in Q_+ and $G = R$ otherwise (S is the set of symbols of T). After the shared and pruned counters, the three-state machine has

$$1 + n + (m_- n + |G|)(n + 1) + n(n + 2 + |R| + |G|) + (n + 2)$$

symbols. Since $|R| \le \min(|\delta|, m_+ n)$, where $|\delta|$ is the number of quintuples of T, this is $O(n(mn + |\delta|)) \subseteq O(mn^2)$, compared with $O(m\, m_+ n^3)$ for Morita's construction. For $(32, 3)$ it gives 659 symbols.

Reflection. Applying the conversion to the mirror image of T (directions L and R exchanged) and mirroring the result back exchanges the roles of m_+ and m_- and of the right-moving and left-moving targets. For $(32, 3)$ this gives 600 symbols, and hence URTM$(3, 600)$. Applied to the other five original machines, the improved conversion gives between 802 and 1,209 symbols (with or without reflection), so $(32, 3)$ is the best starting point.

3.5 Two symbols

Morita's conversion to two symbols [10, Sect. 5.3.3] replaces each symbol by a k-bit block, the blank by 0^k. A quintuple of T is simulated by reading $k - 1$ bits while erasing them, reading the last bit and writing the last bit of the new symbol, writing the remaining bits while moving back, and moving $k - 1$ cells in the direction of the move; the states remember the prefix read or the prefix still to write. If P_r and P_w denote the numbers of read and written prefixes of lengths 1 to $k - 1$, summed over the states, the number of states is at most $mk + P_r + P_w$. From URTM$(24, 4)$ ($k = 2$) this gives URTM$(138, 2)$, and from URTM$(13, 7)$ ($k = 3$) it gives 169 states.

We add four degrees of freedom: the assignment of codes to symbols (the blank fixed to 0^k), the values written in place of the bits that have been read, a path type for each quintuple that says how many bits need not be rewritten on the way back (a right-moving quintuple can then cross its block once instead of three times), and, for each state, the end of the block at which the head stands when reading begins. Reversibility of the result reduces to distinct written values on the rules entering each auxiliary state. For fixed codes and orientations, the written values and the path types are chosen by a constraint programming model that minimizes the number of auxiliary states; the codes and orientations are chosen by exhaustive enumeration and hill climbing. Finally, states with the same behavior are merged; a merged table fills undefined entries of one state with the rules of the other, so it simulates T only along runs that never halt in an undefined entry, which is the case for runs from encoded CTSH inputs. For $(13, 7)$ the

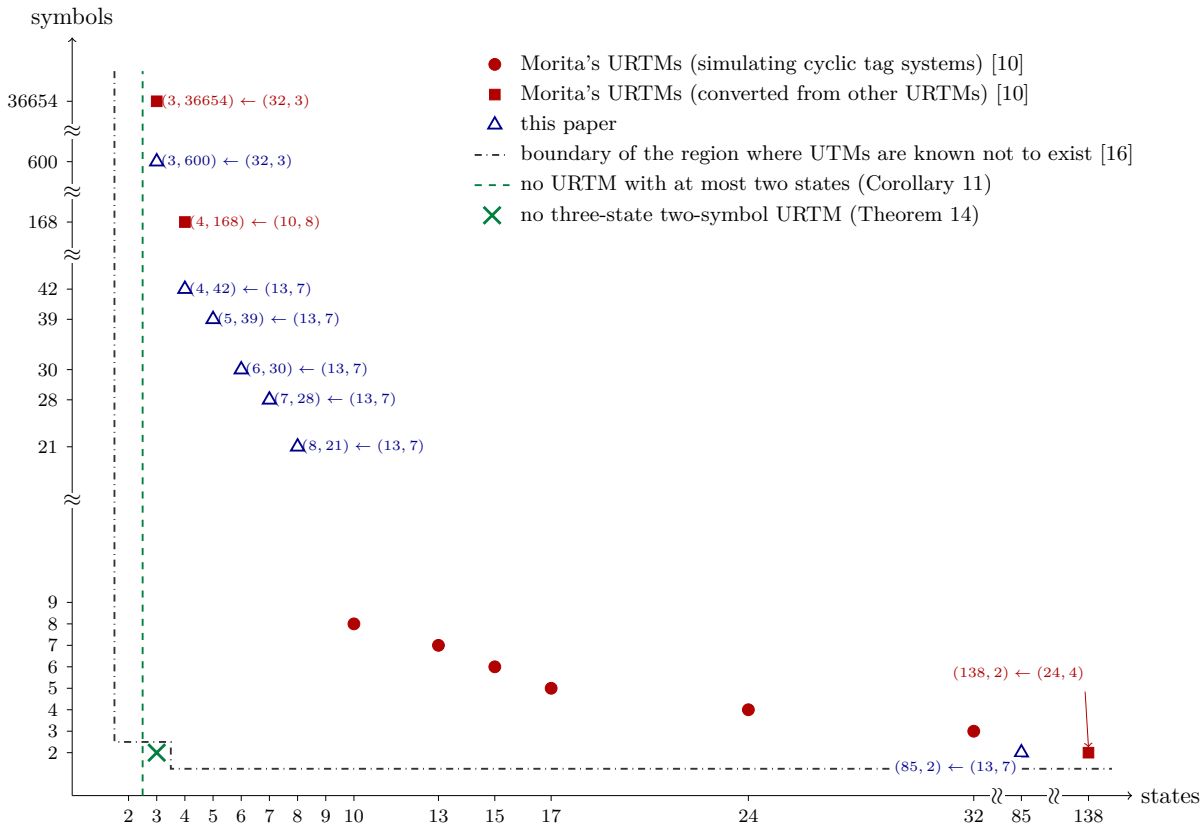


Figure 4: Numbers of states and symbols of URTMs

steps give 149, 135, and 92 states. A randomized large-neighborhood search over the choices of the table before merging then gave a table with 137 states that merges down to 85 states and 153 quintuples, that is, URTM$(85, 2)$. We have no closed formula for the size of this search result; it is verified for the concrete table. Applied to the other five original machines, the same method, including the search, gives 92, 92, 108, 115, and 115 states for $(10, 8)$, $(24, 4)$, $(15, 6)$, $(17, 5)$, and $(32, 3)$. These five tables are also proved universal in the sense of Definition 4, and all of them are dominated by URTM$(85, 2)$.

4 The new URTMs and their universality

Theorem 8 (Existence)**.** *For each of the 13 points*

$$(m, n) \in \{(3, 600), (4, 42), (5, 39), (6, 30), (7, 28), (8, 21), (10, 8), (13, 7), (15, 6), (17, 5), (24, 4), (32, 3), (85, 2)\},$$

there is a table U that is deterministic and reversible, satisfies $\mathrm{NStates}(U) = m$ *and* $\mathrm{NSyms}(U) = n$*, and satisfies* $\mathrm{IsUTMStrict}(U)$*.*

Figure 4 places these points among the known ones. The six points from $(10, 8)$ to $(32, 3)$ are Morita's original machines, which remain non-dominated; the other seven are new. The witnesses are concrete rule tables: Morita's distributed tables [12] and the tables produced from them by the conversions of Section 3.

The dash-dotted line in Figure 4 bounds the region where UTMs are known not to exist [16]: one-state machines, two-state and three-state two-symbol machines (under a definition that reserves one transition for halting), and one-symbol machines. These results concern TMs that

need not be reversible, and their treatment of halting and of zero-shifts differs from ours, so we do not use them for URTMs in our sense. The green lines mark our own nonexistence results (Sections 5 and 6), which are stated for the same notion of universality as Theorem 8.

4.1 Proof of universality

Each witness other than the five merged ones is first shown to simulate an arbitrary CTSH: for every CTSH $\hat{C}$ and initial string v, the machine started on the code of $(\hat{C}, v)$ halts if and only if $\hat{C}$ halts from $(v, 0)$, and its halting ID is the code of the halting ID of $\hat{C}$. For Morita's six machines this is a direct proof about the given tables. For the converted machines it follows from general simulation theorems for the conversions (preservation of determinism, reversibility, one-step simulation, and the correspondence of halting, halting IDs, and non-halting), instantiated with the concrete tables. The five merged machines inherit universality from their tables before merging by Theorem 7.

Universality in the sense of Definition 4 then requires computable maps between arbitrary TMs and CTSHs. We formalize the chain

$$\mathrm{tm}(i) \longrightarrow \text{two-register machine} \longrightarrow \text{2-tag system} \longrightarrow \text{CTSH} \longrightarrow U,$$

which follows the classical route of Cocke and Minsky [4] and Morita's Theorem 7.2. The first step stores the tape of $\mathrm{tm}(i)$ in two registers and normalizes the symbols of $\mathrm{tm}(i)$, which are arbitrary natural numbers, into binary; this replaces a separate conversion of the TM to two symbols. A general theorem then states that a machine that simulates an arbitrary CTSH and has a primitive recursive input encoding and a primitive recursive readout of the final string, independent of trailing blanks, satisfies IsUTMStrict. Each machine supplies only its numbering, encoding, and readout.

Proposition 9 (Agreement)**.** *Let U_1 and U_2 be deterministic tables satisfying* IsUTMStrict *with encodings c_1, c_2 and decodings d_1, d_2. For every TM* $\mathrm{tm}(i)$ *and admissible input a, U_1 halts from* $\mathrm{toID}(0, c_1(i, a))$ *if and only if U_2 halts from* $\mathrm{toID}(0, c_2(i, a))$*, and if both halt, the decoded halting IDs are equal. In particular this holds for any two of the 13 witnesses of Theorem 8.*

The encoded inputs, the runs, and the numbers of steps differ between machines; only halting and the decoded result agree.

Halting quintuples. For the 11 points other than $(85, 2)$ and $(3, 600)$, the distributed tables represent the halting entries of the book by two zero-shift quintuples into halting states that are not counted. We prove Theorem 8 also for the tables without these two quintuples, which are the tables printed in the book or, for the five merged points, their analogues. Removing them changes neither the number of states nor the number of symbols; the decoder first maps the halting ID back to the corresponding halting state.

4.2 Counting states and symbols

Morita does not count halting states in the m of URTM(m, n). The book states that the set of final states is not specified in the problem of reducing the number of states, the state sets of the URTMs in Chapter 7 contain no halting states, and in Chapter 11 it notes that the halting states are not included in URTM$(17, 5)$ [10]. The distributed files also state explicitly that the halting states are not counted.

Our NStates counts source states and NSyms counts the symbols occurring in the table together with the blank. The two conventions could differ for states that occur only as targets, for halting states, and for symbols that do not occur in the table. For the witness tables of Theorem 8 we prove that every target is a source state; for $(85, 2)$ and $(3, 600)$ this holds already

Table 4: Independent count of states and symbols in the witness tables

Table	Quint.	Sources	Target-only	Symbols	Zero-shifts
Morita's 6 points, distributed	59–84	m	2	n	2
same, tables in the book	57–82	m	0	n	0
5 symbol-merging points, distributed	143–162	m	2	n	2
same, without 2 quintuples	141–160	m	0	n	0
$(85, 2)$	153	85	0	2	0
$(3, 600)$	1,572	3	0	600	0

for the tables in the form of the distributed files, and for the other 11 points it holds for the tables without the two halting quintuples. Hence the number of states remains m even under a convention that also counts targets. We also prove that the blank occurs in each witness and that every symbol is both read and written, and that the witnesses have exactly (m, n) under Morita's way of counting (targets counted, only symbols occurring in the table). An independent count outside the proof assistant agrees (Table 4). For the 11 distributed tables with the additional halting quintuples, counting targets gives $m + 2$ states.

5 Nonexistence of URTMs with at most two states

Theorem 10 (Halting of two-state RTMs)**.** *There is a computable Boolean function f such that, for all distinct states q_0, q_1, q_f, every deterministic and reversible table T whose source states are among q_0, q_1 and whose targets are among q_0, q_1, q_f, every position $p \in \mathbb{Z}$, and every code $z \in Code$,*

$$f(T, p, z) = \text{true} \iff \text{Halts}_T(\text{toID}(p, z)).$$

Corollary 11. *If U is deterministic and reversible and $\text{NStates}(U) \leq 2$, then $\neg\text{IsUTM}(U)$, and hence $\neg\text{IsUTMStrict}(U)$. Moreover, there is a TM $\text{tm}(i)$ that no deterministic and reversible two-state table simulates in the sense of conditions 2 and 3 of Definition 4.*

The corollary follows from Theorem 10 and the undecidability of the halting problem. Two reductions bring a general table into the form of the theorem. A one-state table is a special case of a two-state one. If a table has several halting targets, removing the quintuples whose target is not a source state does not change whether the machine halts: if the original run halts by executing a removed quintuple, the reduced run halts one step earlier. Removing quintuples preserves determinism and reversibility. The corollary therefore holds for any number of symbols, and with zero-shifts and halting states allowed as in Morita's definition. If this formalization captures Morita's Definition 7.1, it answers the question of the book and of the survey [11] negatively.

Proof outline of Theorem 10. Let T be as in the theorem, with the quintuples into q_f removed. Fix an initial finite ID and a window $[a, b]$ containing its non-blank cells and its head; let $w = b - a + 1$ and let n be the number of symbols.

1. *The state is the direction of the last move.* By the unique shift-direction property, if T has both a left-moving and a right-moving quintuple, then one state is entered only by left moves, the other only by right moves, and there is no zero-shift. So the state records the direction of the last move, and T is a partial injection $\pi : \{\text{L}, \text{R}\} \times \Sigma \rightharpoonup \Sigma \times \{\text{L}, \text{R}\}$. (If T moves in only one direction, the head enters blank cells beyond the window in one of two states, so a state repeats by the third such cell, the run repeats itself shifted, and it does not halt; a halting run is therefore short.)

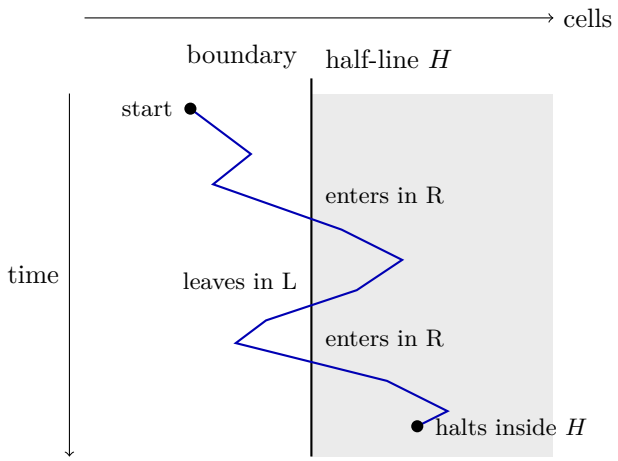


Figure 5: The head of a two-state RTM crossing the boundary of a half-line

2. *A half-line reports only that the head came back.* Fix a boundary and the half-line H to its right. The head always enters H in state R and leaves it in state L, and the behavior inside H does not depend on the content outside. Hence the outcome of the j-th entry into H (leave, halt inside, or never return) and the content of H on leaving depend only on the initial content of H and on j. Figure 5 illustrates this.

3. *The head cannot go deep into a blank half-line.* Let $[k, \infty)$ be initially blank and consider a halting run. The half-line $[k+1, \infty)$ is a translate of $[k, \infty)$, so by step 2 their contents at the j-th entry are translates of each other. If the number of entries into $[k+1, \infty)$ caught up with that into $[k, \infty)$, the run would repeat itself shifted by one cell and never halt. So the numbers of entries into $[k, \infty)$, $[k+1, \infty)$, $[k+2, \infty)$, ... strictly decrease, and the head never passes k plus the number of entries into $[k, \infty)$.

4. *Each cell runs its own clock.* A cell to the right of the initial head is first entered from the left, and afterwards the head can return only from the side to which it left. So the pair (direction, symbol) at the i-th visit of the cell is obtained by iterating a fixed map $\kappa(d, s) = (\overline{d'}, v)$, where $\pi(d, s) = (v, d')$, on a set of size $2n$. All blank cells run the same clock, so a run that halts inside a blank half-line enters it at most $2n + 1$ times.

5. *Inside the window the machine is finite-state.* Replacing each step that leaves the window by a turn at its end gives a machine with at most $Z = 3w\, n^w$ configurations, whose orbit the real run follows inside the window by step 2. A run that halts inside the window does so before the orbit becomes periodic; a run that halts in a half-line enters it in every period, hence by step 4 in boundedly many periods. In both cases each half-line is entered at most $Z(2n + 3) + 1$ times.

6. *A computable bound.* By steps 3 and 5 the head of a halting run stays in a computable interval around the window. A halting run of a deterministic machine never repeats an ID, so it halts within a number of steps bounded by a computable function of n and w. Running T for that many steps decides halting.

Only step 1 uses reversibility; steps 2 to 6 use only determinism and the fact that the state equals the direction of the last move. In an irreversible two-state TM the same state can be entered from both sides, so a half-line can send more than one bit of information, and indeed such machines can be universal. For three states, step 2 fails already for Morita's conversion to three states, which has two right-moving states.

Further nonexistence results.

Theorem 12. *If U is deterministic and* $\mathrm{NSyms}(U) \leq 1$*, then* $\neg\mathrm{IsUTM}(U)$*, even if U is not reversible.*

The tape never changes, so a halting run halts within a bounded number of steps.

Theorem 13. *Let U be deterministic and reversible with at most one left-moving quintuple ℓ, and assume that no zero-shift quintuple has the source state of ℓ as its target. Then* $\neg\mathrm{IsUTM}(U)$*. The same holds with right and left exchanged.*

This is a reversible counterpart of Pavlotskaya's theorem that the halting problem is decidable for two-symbol TMs with a single left-moving instruction [14]; here the number of states and symbols is arbitrary. By reversibility, the only quintuple entering the target of ℓ is ℓ itself.

6 Nonexistence of three-state two-symbol URTMs

Theorem 14. *Let U be deterministic and reversible without repeated quintuples, and suppose there are distinct states q_0, q_1, q_2 and a symbol $\sigma \neq 0$ such that every quintuple of U has its source state among q_0, q_1, q_2 and reads and writes only 0 and σ. Then* $\neg\mathrm{IsUTMStrict}(U)$*.*

The formalized hypotheses are Boolean checks that compare every two entries at different positions of the list, so, unlike the definitions of Section 2, they also reject a table that lists the same quintuple twice. Targets are arbitrary, so halting states and zero-shifts are allowed as in Morita's definition. We first prove the theorem for Morita's formulation on string IDs, with the additional requirement that the encoded inputs use only the symbols 0 and σ. For deterministic two-symbol tables, IsUTMStrict implies this formulation, because the only symbols that occur are the blank and one non-blank symbol; composing the two gives Theorem 14. As a check that the transfer is not vacuous, applying it to the witness for $(85, 2)$ shows that at least one machine satisfies the string-ID form.

Proof outline. Every table is first reduced to a normal form by renaming states and symbols and by removing the quintuples that enter a state other than q_0, q_1, q_2, which changes the halting time by at most one step. A normal form is described by a key, consisting of the codes of its quintuples and the direction in which each state is entered; there are 359,829 keys. For each of them, a decision procedure checked by the kernel either produces a certificate or marks the table as exceptional. A certificate is a finite set of patterns of visits outside a window that is closed under the transitions of the table. It implies, by the pigeonhole principle, that a halting run halts within a computable number of steps, so that halting is decidable and the table is not universal. The exceptional keys are handled by general arguments:

- if a state has no entering quintuple, then after the first step the table is a two-state table, and Theorem 10 applies;
- if every quintuple entering some state comes from that state itself, then a halting run leaves that state within the width of the window plus one steps and never returns;
- if some state is entered by a zero-shift, then that quintuple can be composed with the next one, eliminating the state and again reducing to a two-state machine;

and the remaining keys by arguments for each type of behavior; the last of them are closed by showing that the head of a halting run stays within a bounded distance of the initial window. Counted outside the proof assistant, the exceptional keys come from 2,136 tables in 178 classes, and the last group consists of 10 classes.

7 Formalization

All definitions of Section 2 and all theorems of Sections 3 to 6 are formalized in Lean 4 with Mathlib. Computability in Definition 4 is Mathlib's notion of computable functions. Every theorem is accepted by the Lean kernel, contains no `sorry`, and depends only on the standard axioms (propositional extensionality, the axiom of choice, and the soundness of quotients); in particular, all computations used in proofs are checked by the kernel. The formalization also proves formulas for the numbers of symbols of the conversions as functions of the original table: the block formula, $(2m+1)n$ for Morita's four-state conversion, $(m+1)n$ for disjoint ranges, $N(2M-1)$ for Morita's three-state conversion, the formula of Section 3.4, and the state count of the k-bit two-symbol conversion; the reverse-counting expression is proved as an upper bound. It also proves the exact step counts of Morita's Lemmas 5.9 and 5.10.

Validation against published runs. Morita distributes description files with the rule tables and test runs of his nine URTMs and a Prolog simulator [12]. The formalized machine model reproduces, by kernel computation, the halting step counts of all published test runs. This includes the three runs of URTM$(3, 36654)$, a table with 37,936 quintuples, which take 471,798, 2,348,492, and 57,220 steps; these counts are obtained both by running the table and, independently, from the run of $(32, 3)$ and the exact step counts of Lemmas 5.9 and 5.10, and the kernel checks that the two routes prove the same statement. One published count differs: a test for URTM$(13, 7)$ states 1,073 steps, whereas recomputation and the corresponding file for URTM$(10, 8)$ give 1,173. A simulator and a checker implemented independently of Lean agree with the formalized model on the tested runs.

Trusted base. The kernel guarantees that the proofs are correct relative to the definitions. What remains to be trusted is that the definitions and theorem statements express the intended claims, in particular that Definition 4 faithfully captures Morita's Definition 7.1; this correspondence cannot itself be checked mechanically. We have stated the definitions and the theorems in full in this paper so that the reader can compare them with the original. A further safeguard is that the same definitions are used on both sides: the existence results and the nonexistence results refer to the same notion of universality, so a definition that were trivially true or trivially false for all machines would contradict one of them. The Lean formalization and a full version, which includes further construction details and proofs, are available at `https://github.com/yokoyama-lab/small-machine-zoo`.

8 Conclusion

We have constructed seven new URTMs that, together with six of Morita's machines, give 13 non-dominated points, and we have shown that no URTM exists with at most two states or with three states and two symbols. The new points come from making Morita's conversions economical: numbering by the direction of entry, counting in reverse, splitting state numbers into blocks, sharing and pruning counters, and merging symbols. The two-state result rests on the observation that, in a reversible machine with two states, the state is determined by the direction of the last move.

Several questions remain open. None of the new points is known to be minimal, and the lower and upper bounds are far apart: no URTM with at most two states exists, while the smallest known three-state URTM has 600 symbols. It is open whether some three-state, three-symbol or four-state, two-symbol RTM is universal. For irreversible UTMs, the smallest known machines have far fewer symbols for the same number of states, and it would be interesting to know whether reversibility forces this difference.

Acknowledgments

We are grateful to the late Kenichi Morita, who introduced us to this problem and encouraged us to take it on. This work was supported by JST CREST Grant Number JPMJCR24I4, Japan. Generative AI tools were used in this work, including for computer search, for writing Lean code, and for drafting and translating the manuscript. The author takes responsibility for the content.

A Construction details and intermediate counts

This appendix brings together the construction and proof explanations that accompany the short note. The Lean definitions and theorem statements determine the exact hypotheses and conclusions. The explanations below do not constitute a human inspection of the formalization.

A.1 Four-state and block constructions

Let an original machine have m states and n symbols. Write m_+ and m_- for the numbers of states entered by right and left moves, respectively. Morita's four-state conversion uses $(2m+1)n$ symbols. Giving the two directions separate number ranges reduces the expression to $(m+1)n$. Counting in reverse while carrying a symbol to the left gives the upper bound $(\max(m_+, m_-)+1)n$. Splitting states into blocks, merging roles, and removing unused symbol–counter pairs gives $n + \sum_{i \in V} |X_i|$, where V is the set of counter values used and X_i is the set of original symbols used with value i. For the conversion from $(10, 8)$, the respective symbol counts are 168, 88, 48, and 46. The last value is the block construction before the later symbol-merging improvements.

The corresponding formal statements are collected in `LeanUniv.FinalFormulas`. The expression for reverse counting is an upper bound; the other formulas just listed are equalities for the stated conversion functions. These formulas concern construction families and do not give lower bounds over all universal machines.

For the final five symbol-merging points, the original machine is $(13, 7)$. The recipe files specify the blocks and counter choices, and the merge files specify classes of symbols. The generators reconstruct the tables, while Lean checks their finite properties and applies the general symbol-merging theorem. Rebuilding these files reproduces the selected constructions; it does not repeat the optimization search or establish optimality.

A.2 Three-state conversion

For Morita's three-state conversion, let $M = m_+n + 2n + 2$ and $N = mn^2 + n^2 + 1$ be the sizes of the intermediate machine. The original symbol-count formula is $N(2M - 1)$. Sharing the up and down marks gives NM. A further construction assigns a separate counter limit to each symbol; its count is N plus the sum of these limits.

For the source $(32, 3)$, the successive recorded symbol counts are 36,654, 18,476, 1,820, 659, and 600. The 1,820-symbol intermediate table has only a size calculation: adequacy of its counter limits throughout every run was not proved. The 659-symbol table rebuilds the intermediate machine using one symbol per cell. Reflecting the source machine left to right gives the 600-symbol table. Simulation and universality of the final $(3, 600)$ witness are established by the corresponding Lean theorems.

A.3 Two-symbol conversion and automated search

For a conversion using k-bit codes, let P_r and P_w be the sums, over states, of the numbers of read-code and write-code prefixes of lengths $1, \ldots, k-1$. Let D_0 be the set of source states having a rule whose target is also a source state. The formalized state-count formula is $|D_0| + (k-1)m + P_r + P_w$, bounded above by $mk + P_r + P_w$.

From $(13, 7)$, the recorded state counts are 169 using Morita's code, 149 with a search-selected code, 135 after selecting written values and path types, 92 after state merging, and 85 after the broader automated search. The solver that selected the intermediate tables is not itself a verified Lean conversion function. Its concrete output tables are checked in Lean; the proof for the merged table also establishes the required relationship to its pre-merge table. The relevant intermediate-value theorems are in `LeanUniv.FinalValues`.

The automated search was modeled on Karpathy's autoresearch (`https://github.com/karpathy/autoresearch`). An interactive AI session prepared the search program, fixed evaluator, and instructions at the human's request. After the human started a session, the agent edited the search program and the evaluator checked candidates for determinism, reversibility, and finite CTSH simulations. Only candidates subsequently proved in Lean counted as results. The search found $(6, 37)$, $(8, 28)$, $(9, 27)$, and $(85, 2)$; only the last remains non-dominated after the symbol-merging improvements. The repository includes the construction recipes needed to rebuild the final witnesses.

B Outline of the two-state decidability proof

The formal entry points are `halting_decidable_uniform` and `no_le_two_state_URTM`. This section summarizes the structure of the existing proof; the formal statements specify its scope.

First remove transitions into states with no outgoing transitions by the reduction described in the main text. Let $[a, b]$ contain the initial head position and all initially non-blank cells, and put $w = b - a + 1$. Let Σ contain the blank and every symbol read or written by the table, and let $n = |\Sigma|$. Consider an initial ID whose tape symbols lie in Σ.

1. In a reversible table, transitions entering the same state move in the same direction. If a two-state table has both left and right moves, one state is entered only from the left and the other only from the right; there is no remaining state for a zero-shift target. The state therefore records the direction of the last move. The transition function can be represented by a partial injection on direction–symbol pairs.

2. Consider a half-line to the right of a fixed boundary. Every entry has the right-moving state and every exit the left-moving state. While the head is outside the half-line, its contents do not change. The outcome of the next entry therefore depends on its initial contents and on the number of earlier entries, rather than on the outside tape. The only information passed back to the outside is the return itself.

3. For initially blank nested half-lines, their local computations are translations of one another. In a halting computation, each visited deeper half-line has fewer entries than its predecessor. Otherwise the computation repeats translated to the right and never halts. This bounds the maximum depth by the number of entries at the outer boundary.

4. The sequence of visits to a cell follows a finite local clock on direction–symbol pairs. There are $2n$ such pairs. This bounds by $2n + 1$ the number of entries into a blank half-line if the computation halts inside it. The left-hand case follows by reflection.

5. Replace excursions outside $[a, b]$ by immediate returns at its ends. The resulting dynamics within the window is finite. In the existing proof, $Z = n^w \cdot 3w$ bounds the window states, and $N = Z(2n + 3) + 1$ bounds the relevant entries into each outside half-line. Combining this with the depth bound confines a halting run to $[a - N, b + N]$.

6. That interval contains finitely many IDs over the finite alphabet. A halting run of a deterministic machine cannot repeat an ID. Thus a computable bound on the number of steps suffices to decide halting by bounded simulation. The bound need not be practical.

If only one movement direction occurs, once the head leaves the initial window it repeatedly reads blanks. Repetition of a state gives a translated repetition of the run. A halting run can therefore be bounded in this case as well. The general nonexistence theorem combines the computability of halting for these tables with Mathlib's undecidability of the halting problem.

The first step is where reversibility restricts the two-state model. With three states, a half-line can receive different kinds of entries moving in the same direction; the two-state argument does not apply unchanged.

C Outline of the three-state two-symbol proof

The proof normalizes the tables and checks the 359,829 keys of the normal forms with the Lean kernel. It handles most tables by certificates constraining visits outside a window and by finite-state bounds. Other cases reduce to bounded halting for a two-state machine after removing a state, or require a separate argument about their behavior.

Counts of equivalence classes up to renaming and left–right reflection were computed outside Lean: there are 26,861 such classes in the recorded enumeration, and the last ten classes contain 120 tables. The AI supplied arguments for these final classes, proving that a halting run stays within a bounded neighborhood of the initial window.

The resulting theorem is `URTM.Meta.no_three_two_URTM_all`, followed by the transfer to `no_three_two_URTMStrict_all`. The original target condition is the string-ID form of universality with the stated restriction on encoded symbols. The transfer from `IsUTMStrict` is essential: a proof of nonexistence for a different universality definition alone would not imply the claim made in the note. The finite enumeration files and all imported proof dependencies are included in the repository.

D Comparison with prior work and verification scope

Morita's nine machines provide the baseline [10]. Six of these points remain on the frontier together with the seven new points. The work improves conversion and symbol-assignment choices; it does not prove that the resulting state–symbol pairs are globally smallest. Kudlek and Rogozhin's merging of a blank and another symbol in an ordinary UTM [7] guided the symbol-merging stage. The general theorem here makes the preservation conditions explicit for the formalized universality predicate, including recovery of the symbols on halting tapes.

The lower-bound region quoted for standard UTMs comes from the survey by Woods and Neary [16]. Its conventions for halting and zero-shifts differ from the present model. The new nonexistence claims are therefore established directly in the stated reversible model rather than inferred solely from that comparison. The AI also used prior work on irreversible machines to guide the three-state proof; the author has not checked that correspondence against the original sources.

The supplied generators read Morita's original test configurations from the machine descriptions attached to his 2015 slides. For eight machines, Lean reproduces 44 published halting runs, with the count 1,073 in one $(13, 7)$ comment corrected to 1,173. For $(3, 36654)$, the exact step counts are 471,798, 2,348,492, and 57,220. The large table is checked through a lookup function proved to agree with it and certificates for successive run segments. These finite examples support comparison of the operational model with the supplied data, while the general simulation and universality theorems cover arbitrary encoded inputs.

The two proof environments reuse the specification by copying it. The repository checks that the copies of `Spec.lean` are byte-identical and that the copies of `Strict.lean` differ only in their imports. Each environment is built separately and its imported declarations are audited for nonstandard axioms. These checks establish properties of the formal artifacts; they do not establish the faithfulness of the formalization to the intended mathematical claims.

References

[1] A. Asperti and W. Ricciotti. A formalization of multi-tape Turing machines. *Theoretical Computer Science*, 603:23–42, 2015. https://doi.org/10.1016/j.tcs.2015.07.013.

[2] C. H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. https://doi.org/10.1147/rd.176.0525.

[3] M. Carneiro. Formalizing computability theory via partial recursive functions. In *10th International Conference on Interactive Theorem Proving (ITP 2019)*, LIPIcs 141, pages 12:1–12:17, 2019. https://doi.org/10.4230/LIPIcs.ITP.2019.12.

[4] J. Cocke and M. Minsky. Universality of tag systems with $P = 2$. *Journal of the ACM*, 11(1):15–20, 1964. https://doi.org/10.1145/321203.321206.

[5] A. Dudenhefner. Certified decision procedures for two-counter machines. In *7th International Conference on Formal Structures for Computation and Deduction (FSCD 2022)*, LIPIcs 228, pages 16:1–16:18, 2022. https://doi.org/10.4230/LIPIcs.FSCD.2022.16.

[6] Y. Forster, F. Kunze, and M. Wuttke. Verified programming of Turing machines in Coq. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 114–128, 2020. https://doi.org/10.1145/3372885.3373816.

[7] M. Kudlek and Y. Rogozhin. A universal Turing machine with 3 states and 9 symbols. In *Developments in Language Theory*, Lecture Notes in Computer Science 2295, pages 311–318. Springer, 2002. https://doi.org/10.1007/3-540-46011-X_27.

[8] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science 12699, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[9] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020. https://doi.org/10.1145/3372885.3373824.

[10] K. Morita. *Theory of Reversible Computing*. Springer, 2017. https://doi.org/10.1007/978-4-431-56606-9.

[11] K. Morita. Making reversible computing machines in a reversible cellular space. *Bulletin of the EATCS*, no. 140, 2023. https://bulletin.eatcs.org/index.php/beatcs/article/view/760.

[12] K. Morita. Constructing small universal reversible Turing machines (slides and description files). Hiroshima University Institutional Repository. https://hiroshima.repo.nii.ac.jp/records/2000034.

[13] T. Neary and D. Woods. Four small universal Turing machines. *Fundamenta Informaticae*, 91(1):123–144, 2009. https://doi.org/10.3233/FI-2009-0036.

[14] L. M. Pavlotskaya. Solvability of the halting problem for certain classes of Turing machines. *Mathematical Notes of the Academy of Sciences of the USSR*, 13(6):537–541, 1973. https://doi.org/10.1007/BF01163965.

[15] Y. Rogozhin. Small universal Turing machines. *Theoretical Computer Science*, 168(2):215–240, 1996. https://doi.org/10.1016/S0304-3975(96)00077-1.

[16] D. Woods and T. Neary. The complexity of small universal Turing machines: A survey. *Theoretical Computer Science*, 410(4–5):443–450, 2009. https://doi.org/10.1016/j.tcs.2008.09.051.

[17] J. Xu, X. Zhang, and C. Urban. Mechanising Turing machines and computability theory in Isabelle/HOL. In *Interactive Theorem Proving (ITP 2013)*, Lecture Notes in Computer Science 7998, pages 147–162. Springer, 2013. https://doi.org/10.1007/978-3-642-39634-2_13.